

Diseño y Análisis de Algoritmos

Clase 3: Correctitud de algoritmos

Profesor: Felipe Ruiz

Ayudante: Cristian Nettle

Inducción Matemática

1. Pruebe que para todo número natural m :

$$1 + 2 + \cdots + m = \frac{m(m+1)}{2}.$$

Solución. Probaremos la validez de la fórmula a través de Inducción Matemática:

Base de inducción. Para $m = 1$,

$$1 = \frac{1(1+1)}{2} = \frac{2}{2} = 1.$$

Paso inductivo. Supongamos válida la fórmula para $m = k$ (hipótesis inductiva):

$$1 + 2 + \cdots + k = \frac{k(k+1)}{2}.$$

Entonces, para $m = k + 1$,

$$\begin{aligned} 1 + 2 + \cdots + k + (k+1) &= (1 + 2 + \cdots + k) + (k+1) \\ &= \frac{k(k+1)}{2} + (k+1) \\ &= \frac{k(k+1) + 2(k+1)}{2} \\ &= \frac{k^2 + 3k + 2}{2} = \frac{(k+1)(k+2)}{2} \\ &= \frac{(k+1)((k+1)+1)}{2}. \end{aligned}$$

Queda probado que si vale para k , vale para $k + 1$. Con la base, la igualdad es cierta para todo $m \in \mathbb{N}$.

2. Probar que, para todo número natural m :

$$1^2 + 2^2 + \cdots + m^2 = \frac{m(m+1)(2m+1)}{6}.$$

Solución. Probaremos la validez de la fórmula a través de Inducción Matemática:

Base de Inducción. Para $m = 1$,

$$1^2 = 1 = \frac{1 \cdot 2 \cdot 3}{6}.$$

Paso inductivo. Supongamos válida la fórmula para $m = k$:

$$1^2 + 2^2 + \cdots + k^2 = \frac{k(k+1)(2k+1)}{6}.$$

Para $m = k + 1$,

$$\begin{aligned}
 1^2 + \cdots + k^2 + (k+1)^2 &= \frac{k(k+1)(2k+1)}{6} + (k+1)^2 \\
 &= \frac{k(k+1)(2k+1) + 6(k+1)^2}{6} \\
 &= \frac{(k+1)(k(2k+1) + 6(k+1))}{6} \\
 &= \frac{(k+1)(2k^2 + 7k + 6)}{6} \\
 &= \frac{(k+1)(k+2)(2k+3)}{6} \\
 &= \frac{(k+1)((k+1)+1)(2(k+1)+1)}{6}.
 \end{aligned}$$

Con la base, la igualdad es cierta para todo $m \in \mathbb{N}$.

3. Probar que, para todo número impar n :

$$n^2 - 1 \text{ es divisible entre } 8$$

Solución. Probaremos la validez de la fórmula a través de Inducción Matemática:

Recordatorio. $a \equiv b \pmod{m}$ si $m \mid (a - b)$; es decir, $a - b = km$ para algún $k \in \mathbb{Z}$.

Base de Inducción. Para $n = 1$,

$$1^2 - 1 = 0 \equiv 0 \pmod{8}.$$

Paso inductivo. Supongamos cierta la proposición para el impar $n = 2k + 1$:

$$(2k + 1)^2 - 1 \equiv 0 \pmod{8}.$$

Probemos el siguiente impar $n = 2k + 3$:

$$\begin{aligned}
 (2k + 3)^2 - 1 &= ((2k + 1) + 2)^2 - 1 \\
 &= (2k + 1)^2 + 4(2k + 1) + 4 - 1 \\
 &= ((2k + 1)^2 - 1) + 8k + 8.
 \end{aligned}$$

Por hipótesis inductiva, $(2k + 1)^2 - 1 \equiv 0 \pmod{8}$; y claramente $8k \equiv 0$ y $8 \equiv 0 \pmod{8}$. Luego,

$$(2k + 3)^2 - 1 \equiv 0 + 0 + 0 \equiv 0 \pmod{8}.$$

Con base e inducción, $n^2 - 1 \equiv 0 \pmod{8}$ para todo impar n .

Invariante de loop

4. Considere el *problema de búsqueda*:

Input: Una secuencia A de n números $A = \langle a_1, a_2, \dots, a_n \rangle$ y un valor v .

Output: Un índice i tal que $A[i] = v$, o el valor especial NIL si v no aparece en A .

Escriba pseudocódigo para **Linear-Search**(A, v), que recorre la secuencia buscando v . Usando un **invariante de loop**, pruebe que su algoritmo es correcto. Asegúrese de que su invariante cumpla las tres propiedades necesarias: *inicialización*, *mantenimiento* y *terminación*.

Solución.

Linear-Search(A, v)

```

1  for  $i = 0$  to  $n - 1$ 
2      if  $A[i] = v$ 
3          return  $i$ 
4  return NIL
```

Probaremos la correctitud del algoritmo mediante un invariante de loop.

Invariante $I(i)$. Al inicio de la iteración i del **for** (con $0 \leq i \leq n$):

$$\forall k \in \{0, \dots, i - 1\}, A[k] \neq v.$$

(Equivalente: el prefijo $A[0 \dots i - 1]$ no contiene v .)

Inicialización. Para $i = 0$, el prefijo $A[0 \dots - 1]$ es vacío; la proposición $\forall k \in \emptyset, A[k] \neq v$ es verdadera de manera vacua. Por tanto, $I(0)$ se cumple.

Mantenimiento. Suponga $I(i)$ verdadero al inicio de la iteración i .

- Si $A[i] \neq v$, entonces tampoco aparece v en el prefijo extendido $A[0 \dots i]$. Por lo tanto, al inicio de la siguiente iteración se cumple $I(i + 1)$.
- Si $A[i] = v$, el **for** no continúa; la corrección de este retorno se justifica en **Terminación**.

Terminación. El **for** recorre exactamente los índices $i = 0, \dots, n - 1$; al finalizar, $i = n$. El algoritmo puede terminar de dos formas:

- Si existe $q \in \{0, \dots, n - 1\}$ con $A[q] = v$, entonces en la iteración $i = q$ el algoritmo retorna q . Dado que q es válido y $A[q] = v$, la salida es correcta.
- Si el loop finaliza sin retorno, por $I(n)$ se tiene $\forall k \in \{0, \dots, n - 1\}, A[k] \neq v$; por tanto, v no aparece en A y la salida NIL es correcta.

5. Considere el *problema de la suma acumulada*:

Input: Una secuencia A de n números $A = \langle a_1, a_2, \dots, a_n \rangle$.

Output: El valor $S = \sum_{i=1}^n a_i$.

Escriba pseudocódigo para **Sum**(A), que recorre la secuencia acumulando la suma. Usando un **invariante de loop**, pruebe que su algoritmo es correcto. Asegúrese de que su invariante cumpla las tres propiedades necesarias: *inicialización*, *mantenimiento* y *terminación*.

Solución.

Sum(A)

```

1  answer = 0
2  for i = 0 to n - 1
3      answer = answer + A[i]
4  return answer
```

Probaremos la correctitud del algoritmo mediante un invariante de loop.

Invariante $I(i)$. Al inicio de la iteración i del **for** (con $0 \leq i \leq n$):

$$answer = \sum_{k=0}^{i-1} A[k]$$

(por convenio, la suma vacía es 0).

Inicialización. Para $i = 0$, $answer = 0 = \sum_{k=0}^{-1} A[k]$. Luego $I(0)$ se mantiene.

Mantenimiento. Suponga $I(i)$ verdadero al inicio de la iteración i . En el cuerpo del loop se ejecuta $answer \leftarrow answer + A[i]$. Por tanto, al inicio de la siguiente iteración:

$$answer = \left(\sum_{k=0}^{i-1} A[k] \right) + A[i] = \sum_{k=0}^i A[k],$$

es decir, $I(i+1)$ se mantiene.

Terminación. El **for** recorre exactamente $i = 0, \dots, n-1$; al finalizar, $i = n$. Por $I(n)$, $answer = \sum_{k=0}^{n-1} A[k]$, que es precisamente el valor retornado.

6. Considere el *problema del máximo*:

Input: Una secuencia A de n números $A = \langle a_1, a_2, \dots, a_n \rangle$.

Output: El valor $M = \max\{a_1, a_2, \dots, a_n\}$.

Escriba pseudocódigo para $\text{Max}(A)$, que recorre la secuencia y retorna el mayor valor. Usando un **invariante de loop**, pruebe que su algoritmo es correcto. Asegúrese de que su invariante cumpla las tres propiedades necesarias: *inicialización*, *mantenimiento* y *terminación*.

Solución.

$\text{Max}(A)$

```
1  answer = A[0]
2  for i = 1 to n - 1
3      if A[i] > answer
4          answer = A[i]
5  return answer
```

Probaremos la correctitud del algoritmo mediante un invariante de loop.

Invariante $I(i)$. Al inicio de la iteración i del **for** (con $1 \leq i \leq n$):

$$\text{answer} = \max\{A[0], A[1], \dots, A[i-1]\}.$$

Inicialización. Para $i = 1$, $\text{answer} = A[0] = \max\{A[0]\}$. Luego $I(1)$ se mantiene.

Mantenimiento. Suponga $I(i)$ verdadero al inicio de la iteración i .

- Si $A[i] > \text{answer}$, tras la asignación $\text{answer} \leftarrow A[i]$ se tiene $\text{answer} = \max\{A[0], \dots, A[i]\}$.
- Si $A[i] \leq \text{answer}$, entonces el máximo del prefijo no cambia y $\text{answer} = \max\{A[0], \dots, A[i]\}$.

En ambos casos, al inicio de $i + 1$ se mantiene $I(i + 1)$.

Terminación. El **for** recorre $i = 1, \dots, n - 1$; al finalizar, $i = n$. Por $I(n)$, $\text{answer} = \max\{A[0], \dots, A[n-1]\}$, que es lo que retorna el algoritmo.